Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____
Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking.**

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment — 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                         /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS


    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                 /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare — 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan — 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements.**

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design — 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ANSWERS

TASK 1 – Evaluate the Requirements of Each Computing Environment

A smart hospital requires different operating systems for different devices based on response time, reliability, security and performance.

1. Requirement ranking

Environment	Main requirements	Priority
Patient monitors	Fast response, reliability, safety	Very High
Handheld tablets	Battery life, portability, security	High
Desktop workstations	Performance, multitasking, security	High
Database server	Reliability, availability, data protection	Very High
Imaging server	High processing speed, throughput	High
Partner hospital link	Communication, security, reliability	High

2. Explanation

- Patient monitors: Require real-time operating systems to generate alarms within 10 milliseconds.
- Handheld tablets: Require lightweight mobile operating systems to save battery and support wireless communication.
- Desktop workstations: Require multitasking operating systems for doctors and nurses.
- Database server: Requires high reliability, backup and continuous availability.
- Imaging server: Requires multiprocessing support to process large MRI images.
- Partner hospital link: Requires distributed operating-system services and secure communication.

3. Conflicting requirements

- Fast response may conflict with heavy multitasking.
- High security may require additional processing.
- High throughput may increase resource usage.
- Battery saving may reduce processing performance.

4. Patient safety

The most critical requirement is the response time of patient monitors. An abnormal reading must generate an alarm within 10 milliseconds to avoid delays in emergency treatment.

5. Application of OS history

- Batch systems are useful for scheduled hospital reports.
- Time-sharing systems support multiple hospital users.
- Personal computer systems support doctors' workstations.
- Mobile systems support nurses' handheld tablets.

Alternative Concept and Complexity Analysis

If a single general-purpose OS is used everywhere:

- Alarm response time cannot be guaranteed because normal scheduling may cause delays.
- More memory and storage are required on smaller devices.
- Administration becomes easier because only one OS is maintained.
- However, patient safety and battery efficiency may be affected.

Evaluation: Using different operating systems for different environments is better than using one general-purpose OS because each device has different requirements.

TASK 2 – Judge the Type of Operating System for Each Environment

1. Comparison of OS types

OS Type	Purpose
Mainframe OS	Handles large-scale data processing
Desktop OS	Supports personal computers
Multiprocessor OS	Uses multiple processors
Distributed OS	Connects multiple computers
Clustered OS	Provides high availability
Real-time OS	Provides predictable response time
Handheld OS	Supports portable devices

2. Suitable OS for hospital environments

Environment	Suitable OS type	Reason
Patient monitors	Hard real-time OS	Guarantees alarm deadlines
Handheld tablets	Handheld OS	Saves battery and supports mobility
Workstations	Desktop OS	Supports multitasking
Database server	Clustered OS	Provides fail-over and availability
Imaging server	Multiprocessor OS	Handles parallel processing
Partner hospital link	Distributed OS	Supports data sharing

3. Symmetric and Asymmetric Multiprocessing

- Symmetric Multiprocessing (SMP): All processors share tasks and resources. It provides better load balancing.
- Asymmetric Multiprocessing (AMP): One processor controls the system while others perform assigned tasks.

Evaluation: SMP is more suitable for the MRI imaging server because it distributes processing tasks among multiple processors.

4. Database server clustering

A clustered system uses multiple servers. If one server fails, another server takes over.

This reduces downtime and improves database availability.

Alternative Concept and Complexity Analysis

If one large multiprocessor server is used instead of a cluster:

- Availability decreases because hardware failure can stop the entire server.
- Initial cost may be lower than maintaining multiple servers.
- Throughput increases with more processors, but not always linearly because of shared resources.
- More memory can support larger workloads.

Evaluation: A clustered system is better because it provides fail-over and higher reliability.

TASK 3 – Evaluate Operating-System Structures

1. Types of OS structures

Structure	Explanation
Monolithic	All major OS services run inside one kernel
Layered	OS is divided into different layers
Microkernel	Only essential services run in the kernel
Modular	Supports loadable kernel modules
Hybrid	Combines features of different structures

2. Comparison

- Monolithic: Provides high performance but a faulty driver can affect the entire system.
- Layered: Easy to understand and maintain but communication between layers may add overhead.
- Microkernel: Improves reliability and isolation but message passing can increase overhead.
- Modular: Allows adding or removing modules without rebuilding the entire kernel.
- Hybrid: Combines performance and flexibility.

3. Suitable structures

Patient monitors – Microkernel

- Provides better fault isolation.
- Keeps essential services small.
- Improves reliability.
- Suitable for safety-critical devices.

Database server – Modular kernel

- Supports high performance.
- Allows device drivers and features to be loaded when needed.
- Provides flexibility for updates.

4. Examples

Operating System	Structure
Linux	Monolithic with modular support
Windows	Hybrid
Android	Linux-based modular monolithic kernel
FreeRTOS	Small kernel-based RTOS
QNX	Microkernel

Alternative Concept and Complexity Analysis

If patient monitors use a monolithic kernel instead of a microkernel:

- System calls can be faster because services run inside the kernel.
- Microkernels require message passing between separate services.
- Monolithic kernels may require more kernel memory.
- A faulty driver can crash the complete system.

Evaluation: A microkernel is preferred for patient monitors because reliability and fault isolation are more important than small performance differences.

TASK 4 – Evaluate the Services an Operating System Must Provide

1. Important OS services

OS Service	Purpose
User interface	Allows users to interact with the system
Program execution	Runs hospital applications
I/O operations	Controls input and output devices
File system	Stores and manages files
Communication	Enables data exchange
Error detection	Detects system errors
Resource allocation	Allocates CPU and memory
Accounting	Tracks resource usage
Protection	Prevents unauthorized access
Security	Protects patient information

2. Services required for each environment

Environment	Required services
Patient monitors	Program execution, I/O, error detection, protection
Tablets	User interface, communication, file system, security
Workstations	All major OS services
Database server	File system, communication, protection, security, resource allocation
Imaging server	Program execution, I/O, memory and processor management
Partner link	Communication, protection and security

3. Services that can be removed from patient monitors

- Unnecessary graphical interfaces.
- Unused background applications.
- Unnecessary file-management services.
- Unneeded network services.

This reduces memory usage and improves response time.

4. Essential database services

The database server requires file management, communication, backup, error detection, resource allocation and security.

The OS uses authentication and access permissions to prevent one patient's records from being accessed by unauthorized users.

Alternative Concept and Complexity Analysis

If tablets use a full desktop OS:

- More memory and storage are required.
- Startup time increases.
- Background services consume additional battery power.
- Performance may decrease.

Evaluation: A lightweight handheld OS is better because it provides sufficient services while saving battery and memory.

TASK 5 – Examine System Calls

1. Categories of system calls

Category	Functions
Process control	fork(), exec(), exit()
File management	open(), read(), write(), close()
Device management	ioctl()
Information maintenance	getpid(), time()
Communication	pipe(), socket()
Protection	chmod(), chown()

2. Working of the read() system call

When a program uses read(), the following steps occur:

1. The program calls the read() library function.
2. The system call transfers control to the kernel through a trap.
3. The CPU switches from user mode to kernel mode.
4. The kernel checks the file descriptor and requested operation.
5. The kernel reads data from the file or device.
6. The result is returned to the program.
7. The CPU switches back to user mode.

3. Parameter passing methods

- Registers: Parameters are stored in CPU registers.
- Memory block: Parameters are stored in a memory block.
- Stack: Parameters are passed through the stack.

4. Use of strace

strace is a Linux tool used to monitor system calls made by a program.

Example:
strace -e trace=openat,read,write,close ./program
It displays file operations and their return values.

Alternative Concept and Complexity Analysis
If a program uses fopen(), fread() and fwrite():

- Library buffering reduces the number of system calls.
- A large block of data is transferred in one operation.
- Additional user-space memory is required for buffers.

For a file of \(n\) bytes:

- Buffered system calls: approximately \(n/4096\).
- Small buffer of \(b\) bytes: approximately \(n/b\).

Evaluation: Library buffering is better for large file operations because it reduces system-call overhead.

TASK 6 – Evaluate the Cost of System Calls

1. Why system calls are expensive

A system call is more expensive than an ordinary function call because it involves:

- Switching from user mode to kernel mode.
- Validating parameters.
- Executing kernel operations.
- Returning control to user mode.

2. Effect of buffer size

A larger buffer reduces the number of read() and write() calls.

For example, copying a 10,000-byte file:

Buffer size	Expected read() calls
1 byte	10,001
512 bytes	21
4096 bytes	4

3. Complete C Program

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

long copy_file(const char *src, const char *dst,
               size_t bufsize)
{
   char buf[8192];

   if (bufsize == 0 || bufsize > sizeof(buf))
      return -1;

   int in = open(src, O_RDONLY);
   if (in < 0) return -1;

   int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC,
0644);
   if (out < 0) {
      close(in);
      return -1;
   }

   long calls = 0;
   ssize_t n;

   while (1) {
      n = read(in, buf, bufsize);
      calls++;

      if (n == 0) break;
      if (n < 0) {
```

```
            calls = -1;
            break;
        }

        ssize_t written = 0;

        while (written < n) {
            ssize_t w = write(out, buf + written,
                              n - written);
            if (w <= 0) {
                calls = -1;
                break;
            }
            written += w;
        }

        if (calls == -1) break;
    }

    close(in);
    close(out);

    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");

    if (f == NULL) {
        perror("fopen");
        return 1;
    }

    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);
```

```
    fclose(f);

    size_t sizes[] = {1, 512, 4096};

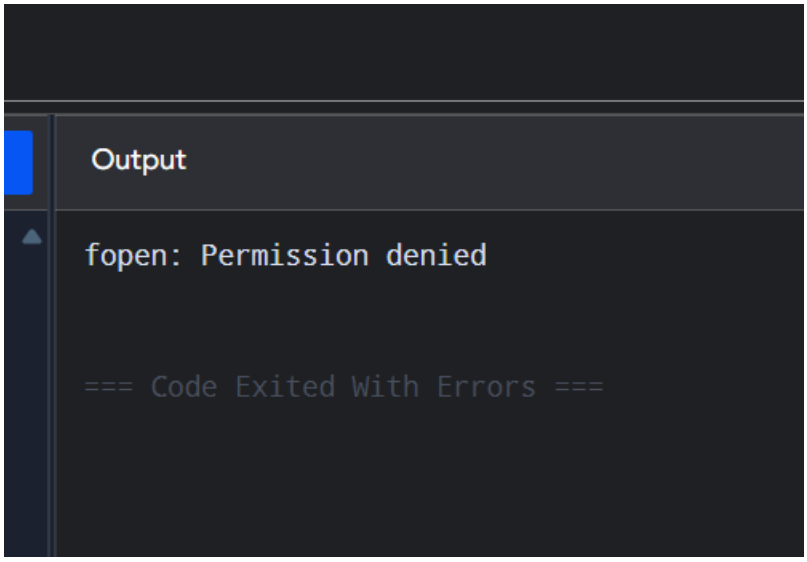
    for (int i = 0; i < 3; i++) {
        long result = copy_file(
            "records.dat", "copy.dat", sizes[i]);

        printf("buffer %4zu bytes: %ld read() calls\n",
               sizes[i], result);
    }

    unlink("records.dat");
    unlink("copy.dat");

    return 0;
}
```

OUTPUT:



5. Performance evaluation

- A 1-byte buffer requires many system calls.
- A 512-byte buffer reduces the number of calls.
- A 4096-byte buffer requires only four read calls.

Recommendation: A 4096-byte buffer is better among the tested sizes because it reduces system-call overhead.

6. Alternative Concept – Memory-Mapped Files (mmap)

If mmap() is used instead of read() and write():

- The file is mapped into the process address space.
- Data can be accessed through memory.
- It reduces explicit read and write system calls.
- It may avoid an extra copy through user-space buffers.
- Large files require careful memory management.
-

Complexity analysis:

- Buffered copying: approximately \(O(n)\) time and \(O(b)\) extra buffer space.
- Memory-mapped copying: approximately \(O(n)\) time, with virtual address space proportional to the mapped file size.

Evaluation: mmap() can be useful for large files, but buffered I/O is simpler and more predictable for the hospital backup program.

TASK 7 – Evaluate Process Control System Calls

1. Functions of process control

Function	Purpose
fork()	Creates a child process
exec()	Replaces the current process with another program
wait()	Waits for a child process
exit()	Terminates a process

2. How a shell executes a command

1. The shell receives a command.
2. It calls fork() to create a child process.
3. The child calls exec() to execute the command.
4. The parent calls waitpid().
5. The child completes execution and returns an exit status.
6. The shell displays the result.

3. Zombie and orphan processes

- Zombie: A child process has terminated, but its parent has not collected its exit status.
- Orphan: A child process continues running after its parent terminates. The OS reparents it to a system process.

4. Complete C Program

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   pid_t pid;
```

```
int status;

printf("shell: running job 1\n");
fflush(stdout);

pid = fork();

if (pid < 0) {
   perror("fork");
   return 1;
}

if (pid == 0) {
   execlp("expr", "expr", "6", "*", "7",
         (char *)NULL);
   perror("execlp");
   _exit(127);
}

waitpid(pid, &status, 0);

if (WIFEXITED(status))
   printf("job 1 exit status %d\n",
         WEXITSTATUS(status));

printf("shell: running job 2\n");
fflush(stdout);

pid = fork();

if (pid < 0) {
   perror("fork");
   return 1;
}

if (pid == 0)
```

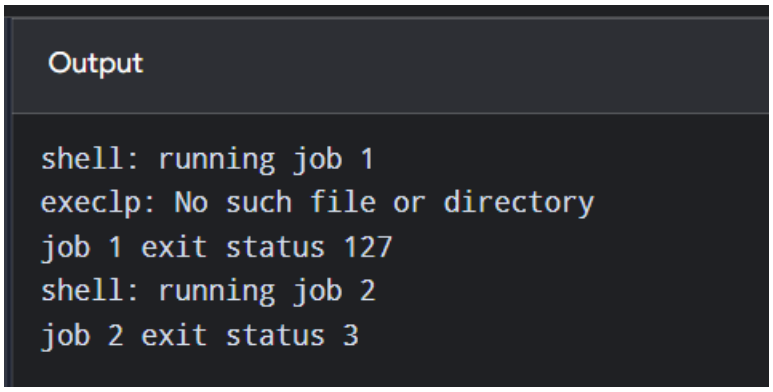
```
        _exit(3);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("job 2 exit status %d\n",
               WEXITSTATUS(status));

    return 0;
}
```

OUTPUT:



6. Why is stdout flushed before fork ()?

When output is buffered, the data may remain in memory instead of being displayed immediately.

After fork(), the child receives a copy of the parent's buffered data. If both processes flush it, duplicate output may appear.

Therefore, fflush(stdout) is used before fork() to prevent duplicate output.

If the second fflush() is removed and _exit(3) is changed to exit(3), redirecting output to a file can cause the buffered message to appear twice.

7. Alternative Concept and Complexity Analysis

Using posix_spawn():

- Creates a process and executes a program through one interface.
- Can reduce process creation overhead in some environments.
- Avoids some of the complexity of separate fork and exec operations.

Using threads:

- Threads share memory and resources.
- They can be faster to create.
- A faulty thread can affect the entire process.

Factor	**Processes**	**Threads**
Memory	Separate address spaces	Shared address space
Isolation	High	Lower
Creation overhead	Higher	Lower
Communication	IPC required	Shared memory

Evaluation: Processes are better for isolating hospital commands, while threads are suitable for tasks that need shared resources.

TASK 8 – Evaluate CPU States and Protection

1. User mode and kernel mode

- User mode: Applications run with limited permissions.
- Kernel mode: The OS runs with full access to hardware and system resources.
- Mode bit: Indicates whether the CPU is operating in user mode or kernel mode.
-

2. Privileged instructions

Privileged instructions can only be executed in kernel mode. Examples include disabling interrupts and controlling hardware devices.

3. Interrupts and traps

- Interrupt: A signal that informs the CPU about an event.
- Trap: A software-generated event used for system calls or exceptions.
- Timer interrupt: Allows the OS to regain CPU control and schedule another process.

4. Hardware protection

- Prevents user programs from accessing kernel memory.
- Protects one program's memory from another.
- Prevents unauthorized execution of privileged instructions.
- Ensures that only authorized OS operations control hardware.

If any program could disable interrupts, patient-monitor alarms could be delayed, affecting patient safety.

5. Alternative Concept – Polling

Polling means continuously checking a sensor for new readings.

Interrupts	**Polling**
CPU responds when an event occurs	CPU checks at regular intervals
Less CPU wastage	More CPU usage
Faster event notification	Delay depends on polling interval
Better power efficiency	May consume more power

Worst-case delay:

If the polling interval is 5 milliseconds, an abnormal reading may take nearly 5 milliseconds to be detected, excluding processing time.

Interrupt-based detection usually provides lower event-detection delay.

Evaluation: Interrupt-driven I/O is preferred for patient monitors because it provides faster detection and reduces unnecessary CPU usage.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

1. Memory hierarchy

Memory hierarchy arranges storage according to speed, capacity and cost.

Memory	Speed	Capacity
Registers	Fastest	Very small
Cache	Very fast	Small
RAM	Fast	Medium
SSD	Slower than RAM	Large
Hard disk	Slower	Very large
Network storage	Depends on network	Very large

2. Suitable data placement

Data	Storage location	Reason
MRI images	SSD / Network storage	Large image storage
Patient records	Database server SSD	Fast access and reliability
Monitor readings	RAM and persistent storage	Fast access and permanent records
Frequently accessed records	Cache / RAM	Faster retrieval

3. Types of I/O

Programmed I/O

- CPU continuously checks device status.
- Simple but wastes CPU time.

Interrupt-driven I/O

- Device sends an interrupt when it needs attention.
- Reduces unnecessary CPU checking.

Direct Memory Access (DMA)

- Transfers data directly between a device and memory.

- Reduces CPU involvement.

4. Suitable I/O for MRI scanner

DMA is suitable for MRI scanners because MRI images are large and require high-speed data transfer.

Caching improves record lookup by keeping frequently accessed data in faster memory.

5. Alternative Concept – Interrupt-driven I/O instead of DMA

If a 500 MB MRI image is transferred using one interrupt per word, the number of interrupts will be extremely high.

This increases CPU overhead and reduces performance.

DMA	**Interrupt-driven I/O**
Transfers large blocks	May interrupt for every word
Less CPU involvement	More CPU involvement
High transfer efficiency	Higher processing overhead
Requires DMA buffers	Requires data buffers

Evaluation: DMA is better because it transfers large MRI images efficiently with fewer CPU interruptions.

TASK 10 – Measure and Compare

1. Purpose

This task measures system-call counts, execution time and the performance of processes and threads.

2. System-call measurement

Execute the Task 6 program using different buffer sizes.
Linux commands:
gcc task6.c -o task6

./task6

strace -c ./task6

3. Measure execution time

time ./task6
The time command displays:

- Real time: Total elapsed time.
- User time: Time spent executing user-level code.
- System time: Time spent executing kernel operations.

4. Process creation measurement

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main(void)
{
   clock_t start = clock();

   for (int i = 0; i < 1000; i++) {
```

```
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            return 1;
        }

        if (pid == 0)
            _exit(0);

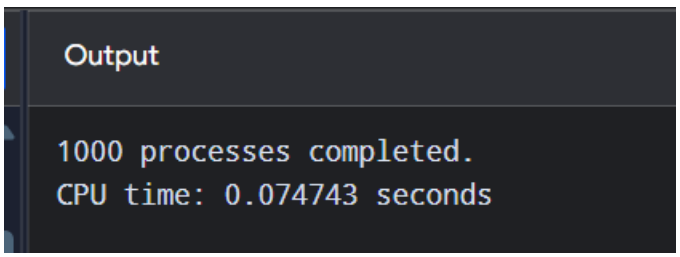
        waitpid(pid, NULL, 0);
    }

    clock_t end = clock();

    printf("1000 processes completed.\n");
    printf("CPU time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    return 0;
}
```

OUTPUT:



5. Thread creation measurement

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>

void *work(void *arg)
{
   return NULL;
}

int main(void)
{
   pthread_t t[1000];

   clock_t start = clock();

   for (int i = 0; i < 1000; i++) {
      if (pthread_create(&t[i], NULL,
                         work, NULL) != 0) {
         printf("Thread creation failed\n");
         return 1;
      }
   }

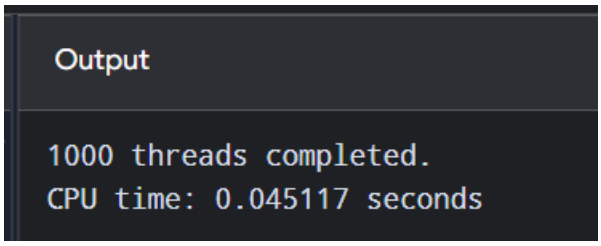
   for (int i = 0; i < 1000; i++)
      pthread_join(t[i], NULL);

   clock_t end = clock();

   printf("1000 threads completed.\n");
   printf("CPU time: %.6f seconds\n",
         (double)(end - start) / CLOCKS_PER_SEC);

   return 0;
}
```

OUTPUT:



6. Measurement comparison

Record your actual results in this table:

Measurement	Result
Buffer 1 byte	10,001 read calls
Buffer 512 bytes	21 read calls
Buffer 4096 bytes	4 read calls
Time for 1000 processes	Record actual output
Time for 1000 threads	Record actual output

7. Alternative Concept and Complexity Analysis

If threads are created instead of processes:

- Thread creation generally requires fewer resources.
- Threads share the same process memory.
- Processes have separate address spaces.
- Processes provide better isolation.

Time complexity: Both experiments perform 1,000 creation operations, so the number of operations is \(O(n)\).

Space complexity:

- Processes require separate address spaces and additional OS resources.
- Threads share process memory but require individual thread stacks and control structures.

Evaluation: Threads generally use fewer resources, while processes provide better isolation. Actual performance depends on the Linux system and configuration.

TASK 11 – Recommend and Defend the Operating-System Plan

1. Recommended OS plan

Hospital environment	OS type	OS structure
Patient monitors	Hard real-time OS	Microkernel
Handheld tablets	Handheld OS	Mobile kernel
Workstations	Desktop OS	Hybrid / modular
Database server	Clustered OS	Modular
Imaging server	Multiprocessor OS	Modular
Partner hospitals	Distributed system	Distributed services

2. Required services

- Patient monitors require real-time scheduling, device control and error detection.
- Tablets require wireless communication, battery management and security.
- Workstations require multitasking and file management.
- Database servers require backup, authentication and fail-over.
- Imaging servers require multiprocessing and DMA.
- Partner hospitals require secure network communication and data sharing.

3. Protection mechanisms

- User authentication.
- Access control.
- Memory protection.
- Data encryption.
- Regular backups.
- Network security.

4. Limitations and risks

- Different OS platforms increase maintenance complexity.
- Hardware failures can affect hospital services.
- Network failures may interrupt data sharing.
- Security vulnerabilities may expose patient records.
- Regular updates and monitoring are required.

5. Support for three hospitals

The plan can support three hospitals using distributed services and secure communication.
Each hospital can maintain its local services while sharing patient records through a central or replicated database system.

6. Alternative Concept – Virtual Machines

If all systems run as virtual machines on two large servers:
Advantages:

- Better hardware utilization.
- Easier backup and management.
- Multiple OS environments can run on the same hardware.

Disadvantages:

- Virtualization introduces some processing overhead.
- Hardware failure may affect multiple virtual machines.
- Real-time alarm response may become unpredictable.
- Additional memory and CPU resources are required.

Evaluation: Separate physical or dedicated real-time systems should be retained for patient monitors. Virtual machines can be used for non-critical applications such as testing, reporting and some server workloads.

TASK 12 – Re-evaluate and Adapt the Design

Changed requirement: The hospital introduces remote patient-monitoring devices in patients' homes and moves patient records to a cloud service shared with partner hospitals.

A. Impact Analysis (2 Marks)

The new requirements affect the original operating-system design in the following ways:

1. Home monitoring devices require lightweight embedded operating systems.
2. Real-time monitoring is necessary to detect abnormal readings quickly.
3. Cloud-based records require distributed-system support.
4. Internet communication requires authentication and encryption.
5. Patient information must be protected from unauthorized access.
6. Network failures must be handled without losing important patient readings.

Evaluation: The original design needs additional support for remote devices, cloud services and secure communication.

B. Evaluation of the Original Design (2 Marks)

The original design provides good performance and reliability inside the hospital.

However, some limitations exist:

- Patient monitors are designed mainly for hospital environments.
- Home devices require wireless and internet connectivity.
- Local databases cannot independently provide cloud-based sharing.
- Internet failures may delay communication.
- Additional security is required for remote access.

Evaluation: The original design is partially suitable but needs modifications to support the changed requirements.

C. Adaptation or Redesign (2 Marks)

The revised design includes the following:

Environment	Recommended solution
Home monitoring devices	Embedded real-time OS
Hospital monitors	Hard real-time OS
Handheld tablets	Mobile OS
Doctor workstations	Desktop OS
Patient records	Cloud-based database
Partner hospitals	Distributed system
MRI processing	Multiprocessor OS

Additional features:

1. Secure communication between home devices and hospital servers.
2. Cloud storage for patient records.
3. Data encryption during transmission and storage.
4. Automatic backup and recovery.
5. Authentication for doctors and nurses.
6. Local storage of readings during internet failure.

D. Evidence-Based Defence (2 Marks)

I recommend MODIFYING the original design instead of completely replacing it.

Reasons:

- The existing real-time OS continues to provide fast monitoring.
- Embedded OS supports remote monitoring devices.
- Cloud services improve data accessibility.
- Distributed systems support sharing records between hospitals.
- Encryption and authentication improve patient-data security.
- Backup and recovery improve reliability.

Trade-offs:

- Cloud services increase network dependency.
- Additional security increases system complexity.
- Cloud storage and communication increase operational costs.
- Local buffering is needed to prevent data loss during internet failure.

Final Evaluation: Modifying the original design is the best solution because it maintains patient safety and existing hospital performance while supporting home monitoring and cloud-based patient records.